

This is the complete, self-sufficient, and unambiguous master requirements and implementation document for **Cage Brawlers**. It consolidates every mechanic, mathematical edge-case, data schema, and networking constraint outlined in your Overview.md file along with our design alignment for a 2D isometric, border-based tactical engine.

Cage Brawlers: Technical Specification & Implementation Manual

Phase 1: High-Level Client-Server Architecture

To eliminate structural ambiguity and prevent map-hacking, the game utilizes a strict server-authoritative architecture built on top of Godot 4's high-level networking features.

1.1 Server State & Network Topology

- **Dedicated Headless Server Mode:** The game is compiled to run as a headless linux server executing matching instances via a custom NetworkManager.gd autoload node using ENetMultiplayerPeer.
- **The Dumb Client Pattern:** The client binary handles only user input gathering, user interface rendering, sound/visual asset loading, and interpolation of states received via Remote Procedure Calls (RPCs). The client maintains no local variables regarding target HP, positioning, or item inventory that can alter game outcomes.
- **Handshake and Roster Locking:** * When a player connects, a handshake protocol verifies their player profile from a global account database.
 - When a player joins a match queue, their active team layout array is validated: it must contain exactly TEAM_SIZE = 3 living characters.
 - Once a match is made, the server locks these character configurations, preventing item swapping or stat changes during active gameplay.

Phase 2: Data Architecture & Core Game Systems

All data schemas must be built using custom serialized Godot Resource objects (class_name) to easily synchronize state fields between the server database and the client interface.

2.1 The Complete Character Resource Model (CharacterData.gd)

Every character instance tracks the following distinct, structural variables:

- **Identity:** Unique ID, character Name, current Level, current Experience points.
- **State Flag:** LIVING, PENDING_LEVEL_UP, KNOCKED_DOWN, or DEAD.
- **Primary Core Stats:**
 - strength (Int): Directly scales physical attack damage and total carry weight capacity.
 - dexterity (Int): Directly scales physical accuracy, base movement speed, evasion multipliers, and initiative rolls.
 - constitution (Int): Directly determines maximum health calculation parameters.
 - wisdom (Int): Dictates ranged attack damage, class-dependent magic damage, healing capabilities, and specific magical skill modifiers.

- intelligence (Int): Dictates ranged attack accuracy, class-dependent magic damage, healing capabilities, and specific magical skill modifiers.

2.2 Dynamic Combat Stat Derivations & Calculations

The engine calculates active derived metrics using precise structural math hooks. Implement these formulas inside the CharacterData.gd resource:

Max Weight Capacity & Encumbrance Logic

- Calculate the maximum carry limit using a scaling curve tied to the character's strength stat.
- Evaluate total equipped burden by summing the hardcoded item weights across all slots: Main Hand, Off Hand, Armor, Belt, and Ammo Pockets.
- **Encumbrance Formula Engine:** Run a check before every move action:

$$\text{Excess Weight} = \text{Total Burden Weight} - \text{Max Carry Weight Limit}$$
 - If $\text{Excess Weight} \leq 0$, apply no penalties.
 - If $\text{Excess Weight} > 0$, the character becomes **Overencumbered**:

$$\text{Movement Speed Penalty} = 1 + \left\lfloor \frac{\text{Excess Weight}}{3} \right\rfloor$$
 Subtract this value directly from the character's base movement speed pool.

Evasion Rule Gating Engine

Evasion is calculated by isolating character attributes from active protective items to prevent negative feedback errors.

- **Character Evasion:** Calculated from the character's dexterity stat and altered by active armor modifiers. If the result drops below 0, clamp it to 0.
- **Equipment Evasion:** Derived by summing the innate defense values of shields and external buff spells. If the result drops below 0, clamp it to 0.
- **Total Combined Evasion:** Sum the clamped components:

$$\text{Final Defender Evasion} = \max(0, \text{Character Evasion}) + \max(0, \text{Equipment Evasion})$$
 Pass this absolute value to combat targets to evaluate incoming accuracy.

Percentage-Based Segmented Health Bars

- Core health pool size is calculated directly from the character's constitution stat.
- Every character data profile must maintain a 3-element float array representing their health segments.
- **Structural Allocation:** The exact size of each segment is a hardcoded percentage distribution defined by the character's Class, rather than absolute flat numbers:

$$\text{Segment}_0 = \text{Max HP} \times \text{ClassPercentage}_0$$

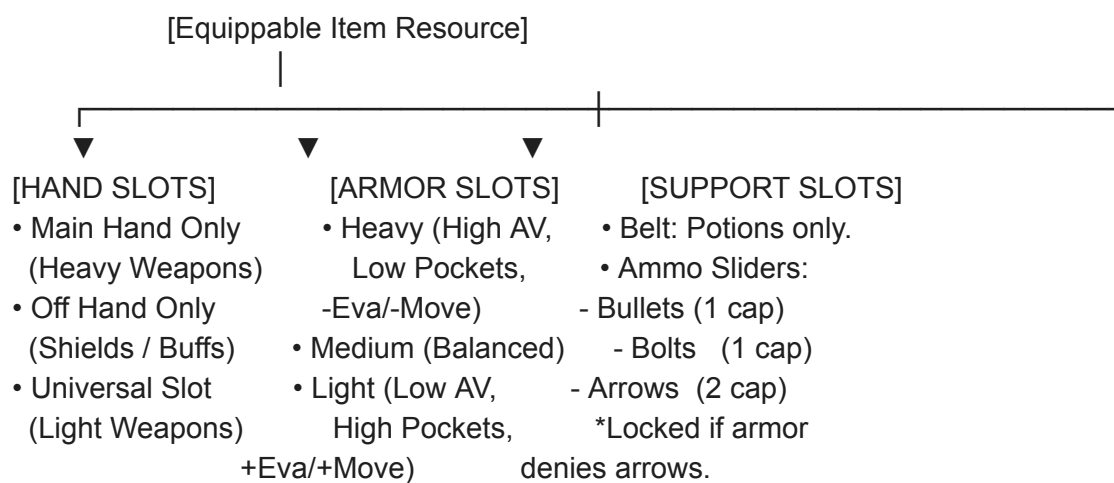
$$\text{Segment}_1 = \text{Max HP} \times \text{ClassPercentage}_1$$

$$\text{Segment}_2 = \text{Max HP} \times \text{ClassPercentage}_2$$
- **Spillover Damage Resolution:** Damage tracking must ignore segment boundary gates during active combat rolls. If a character takes damage greater than the current active

segment's remaining capacity, subtract the remainder directly from the next adjacent segment down the array.

2.3 Equipment & Inventory Inventory Matrices

Equipped items are strictly locked during combat and handled via a secure out-of-match lobby interface.



Hand Equipment Assignment Blueprint

- Every character holds two item-pointer variables: `main_hand_slot` and `off_hand_slot`.
- **Weapons Type Rules:** Heavy weapons are classified as Main Hand only. Shields and Buffs are strictly classified as Off Hand only.
- **The Light Weapon Exception:** Items possessing the Light keyword bypass main-hand restrictions and can be assigned into the `off_hand_slot`, allowing for valid dual-wield setups.
- **Two-Handed Lockout Check:** If an item contains the Two-handed keyword, the validation system maps it to the `main_hand_slot` and automatically sets `off_hand_slot` = NULL while disabling secondary weapon selection fields.

Armor Classification & Support Slot Modifiers

Armor models define character capacity boundaries, serving as the baseline for ammunition limits.

- **Heavy Armor:** High Armor Value, structural reductions to character evasion and base movement speed variables, and sets `ammo_slots_capacity` = 0 or extremely low thresholds with zero potion belt slots.

- **Medium Armor:** Standard baseline values for armor and movement, opening normal configurations for support gear.
- **Light Armor:** Low or no Armor Value, adds positive bonuses to base mobility and character evasion stats, and opens maximum configurations for consumable slots.
- **The Ammo Slider Engine:** * Support item capacity is mapped using responsive slider input nodes in the lobby roster configuration window.
 - Summed capacity calculations are processed via strict item weights:

$$\text{Allocated Space} = (\text{Count}_{\text{Bullets}} \times 1) + (\text{Count}_{\text{Bolts}} \times 1) + (\text{Count}_{\text{Arrows}} \times 2)$$
 - Enforce immediate clamping: Allocated Space cannot exceed the active armor's ammo_slots_capacity variable.
 - Check compatibility: if the armor resource flag allows_arrows == false, force the Arrows slider to remain locked at zero.

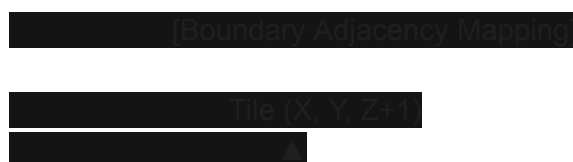
2.4 Procedural Skill Tree Generation

Skills are managed through procedural, seeded node charts calculated at the moment a character asset is initialized.

- **The Three Tree Structure:** Each character contains exactly three distinct skill trees saved within their user data file:
 1. **Class Tree:** Populated by archetypal skills linked to the character's primary chosen Class identity.
 2. **Attribute Tree:** Generated dynamically by checking the character's highest core primary stat variable at creation time.
 3. **Weapon Tree:** Populated by random weapon skills chosen from a sublist of weapons allowed by the character's Class.
- **Randomized Balancing Rule:** The progression pathways and connecting nodes are randomly sorted and shuffled during generation. To maintain systemic balance, skills must be mapped into strict Level requirement groupings so that high-tier abilities never appear early in the tree pathing arrays.

Phase 3: Pseudo-3D Isometric Map Data Topology

The layout balances 2D isometric front-end displays with an authoritative 3D server map array to manage height mechanics.





3.1 Adjacency Grid Graph Infrastructure

- **Grid Mapping Units:** The map array acts as an explicit coordinate network where every distinct tile cell is indexed as a 3D coordinate vector: $\text{Vector3i}(x, y, z)$. Here, z represents the floor level.
- **The Border Tracking Data Schema:** Tiles do not contain built-in physical collision walls. Instead, coordinate boundaries are stored in a centralized map layout lookup table. Each boundary connection contains three boolean flags: `has_wall`, `has_barricade`, and `has_ladder`.
- **Client-Side Isometric Mapping Hooks:** The client loads a standard 2D `TileMapLayer` object configured to use Godot's built-in isometric diamond coordinate orientation. Boundary decorations (walls, barricades, ladders) are placed as separate asset scenes instantiated at runtime with structural offsets matching cell edge locations.

3.2 Pathfinding Logic Extensions (AStar3D)

Movement operations calculate complex path routing by applying custom edge weights across the grid network:

- Map topology connections are mapped using an instance of Godot's AStar3D routing library.
- **Boundary Evaluation Loop:** During level initialization, the server loops through all adjacent tile nodes and evaluates their shared boundaries:
 - If `has_wall == true`, do **not** generate a pathfinding link between those two tiles.
 - If `has_barricade == true`, drop the default traversal connection link (unless a specialized navigation skill enables vault actions).
 - **The Ladder Connection Rule:** If a boundary linking a tile at layer z to a tile directly above it at layer $z+1$ contains `has_ladder == true`, generate a custom pathfinding connection between them. Override the traversal cost to apply an extra movement penalty modifier (e.g., climbing costs $+1$ additional movement step).

Phase 4: Core Combat Loop & Targeting Verification

4.1 Recursive Initiative Placement Resolution

- At the beginning of a match round, every character runs an initiative roll calculation.
- **The Tie-Breaker Resolution Engine:** If two or more characters return identical initiative values, the server executes a recursive evaluation loop:
 1. Isolate the conflicting combatants and trigger a distinct secondary rolling event.

2. The winner of this tie-breaker roll receives a temporary \$+1\$ bonus modifier added to their final initiative placement ranking.
3. **The Collision Check Recursion:** Re-evaluate the entire combat order array from the beginning to ensure that adding the \$+1\$ modifier has not created a new conflict with a different character's initiative value.
4. Repeat this step recursively until every combatant holds a completely unique turn order index.

4.2 Action Phase Container Architecture

A character's turn is structured as three discrete, sequential phase wrappers rather than a generic action points pool.

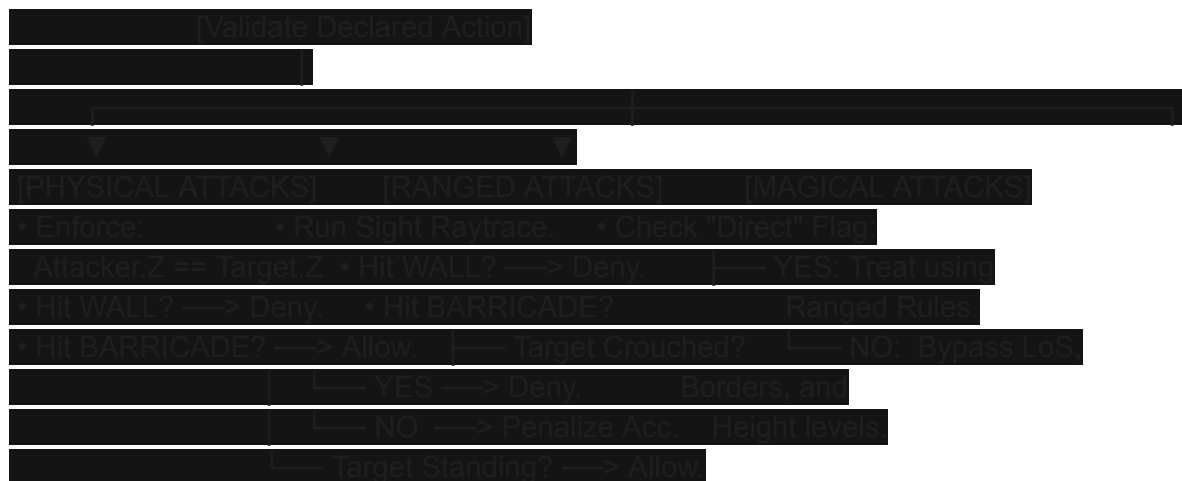
- **Beginning Action Phase:** Dedicated to movement actions and character stance setups. Characters can use their base movement capabilities or swap out the phase to execute specialized beginning abilities.
- **Main Action Phase:** Dedicated to standard attack declarations and executing active offensive skills.
- **Ending Action Phase:** Dedicated to low-impact utility steps, defensive positioning tools, and stance adjustments (such as crouching behind an adjacent barricade).

4.3 Directional Sight Engine (Authoritative Line of Sight)

Characters track directional visibility fields across 8 distinct angles mapped in 45-degree steps.

- **Mandatory Turning Sub-Phase:** Upon completing a move execution inside the Beginning Action Phase, the server freezes the character's state machine into a `PENDING_ROTATION` sub-state. The client UI must display an overlay with 8 directional arrows. The player must select a direction to update their character's `facing_direction` variable before the server opens the Main Action Phase.
- **The Sight Mapping Routine:** The server calculates a character's visibility map by combining two distinct vision zones:
 1. **The Proximity Bubble:** Every tile cell within an absolute 2-tile radius (using Chebyshev distance calculation) is automatically marked as visible, completely bypassing walls and orientation angles.
 2. **The 90-Degree Vision Cone:** Project a 90-degree vision field vector originating from the character's cell coordinate and extending outward along their current `facing_direction` line.
- **Boundary Raytracing Rules:** Trace a path vector from the observer's cell to each target cell within the vision cone:
 - If the vector crosses a boundary where `has_wall == true`, terminate the ray instantly (Sight Blocked).
 - If the vector crosses a boundary where `has_barricade == true`, evaluate the target entity's state flags: if `is_crouched == true`, terminate the ray and hide the target (Sight Blocked).
 - **Adjacency Override:** If the observer is positioned on a tile directly adjacent to the opposite side of that exact barricade boundary, bypass the crouch check and grant full visibility of the target.

4.4 Comprehensive Attack Validation & Keyword Framework



Physical (Melee) Attacks

- Validate height equity: Compare `attacker.grid_position.z` against `target.grid_position.z`. If the floor values are different, reject the action packet.
- Check boundary obstruction: If any intermediate boundary line between the attacker and the target cell has `has_wall == true`, reject the action packet. If a boundary has `has_barricade == true`, allow the melee attack to process normally.

Ranged Attacks

- Bypass the vertical floor height equity check entirely, allowing targeting across different structures and elevations.
- Run the Ranged Target Path-Checker: project a targeting vector from the attacker to the target cell.
 - If it hits a WALL boundary, invalidate the attack.
 - If it hits a BARRICADE boundary while the defender has `is_crouched == true`, invalidate the attack (Target Protected).
 - If it hits a BARRICADE boundary while the defender has `is_crouched == false`, approve the target as valid, but apply a permanent **Cover Accuracy Penalty** modifier to the upcoming accuracy calculation roll.

Magical Attacks

- **The Default Magic Rule:** By default, standard magical spells completely bypass the

line-of-sight raytracer, cell boundary restrictions, crouching protections, and vertical structural elevations. The target is validated unconditionally as long as it falls within the skill's absolute range radius.

- **The Direct Keyword Intercept:** If a magical skill contains the hardcoded Direct keyword modifier, disable the default magic targeting rules. The spell must be evaluated using the strict **Ranged Attacks** verification pathing, requiring clear line of sight and applying accuracy penalties for un-crouched barricade cover.

4.5 The Crouch State Engine

- **Activation Parameters:** A character can declare a Crouch action during their Ending Action Phase container. The server verifies that at least one of the 4 immediate boundaries surrounding the character's current cell contains `has_barricade == true`; if no barricade is found, the action packet is rejected.
- **The Free Stand Up Action:** A character sitting in the `is_crouched == true` state can trigger a **Stand Up** action at any point during their Beginning Action Phase container. This resets `is_crouched = false` instantly on the server without consuming an action phase slot.
- **Mobility Restrictions:** If a character starts a movement action while their state flag is `is_crouched == true`, their calculated base movement speed pool is cut in half for that action.
- **The Combat Break Intercept:** A crouched character can look over and fire attacks across their adjacent barricade boundary. However, if a character initiates *any* attack sequence (Physical, Ranged, or Magical) while `is_crouched == true`, the server must force `is_crouched = false` *before* processing accuracy or damage rolls. This instantly updates the character's visibility state across all clients.

4.6 Dual-Wield Attack Resolution & Resource Consumption

- When a character dual-wielding valid light weapons declares an attack action, the server processes the execution as two separate, independent attack rolls fired in immediate succession.
- **Ammunition Gated Intercept Loop:**
 1. Initialize the loop wrapper for Attack 1 (Main Hand).
 2. Query the ammunition type required by the Main Hand weapon from the armor's pocket array. If the ammo count is `$ > 0$`, decrement the pool by 1 and evaluate the attack hit/damage resolution.
 3. Initialize the loop wrapper for Attack 2 (Off Hand).
 4. Query the ammunition type required by the Off Hand weapon. If the ammo count has reached 0, terminate the attack sequence immediately. Cancel Attack 2's execution without rolling dice, and send an RPC packet to update the client UI with a "No Ammo" warning.

4.7 Multi-Dice Rolling & Reliability Modifier Function

- Implement a core utility function `roll_dice(dice_count: int, dice_sides: int, uses_reliability:`

bool, reliability_value: float) -> int.

- The function loops through the dice_count parameter to roll multiple individual dice instances and bundle their total values together (e.g., evaluating a 2d6 damage resource payload).
- If uses_reliability == true, intercept the final random roll outcome and apply your custom math formula using the character's *Reliability* variable before passing the value back to the damage system.

4.8 Segmented Health Damage & Knockdown Lifecycle

- **The Knockdown Trigger:** When a character's combined health segments drop to exactly 0 HP, pause damage processing and switch their active state flag to KNOCKED_DOWN.
- **Segment Disabling Rule:** Identify the character's right-most, non-disabled health array slot and flag it as permanently DISABLED for the remainder of the match. This permanently reduces the character's maximum possible health pool for that battle.
- **Knocked Down State Behaviors:** A knocked-down character is completely removed from the turn initiative order list, cannot execute actions, and cannot move across cells. They remain stationary on their tile until an ally executes a revival skill.
- **The Permanent Death Condition:** If a character takes damage that reduces them to 0 HP while their final remaining health array segment is flagged as disabled, switch their state flag to DEAD.

Phase 5: Match Lifecycle & Economy Integration

5.1 Match End Boundaries

A match session is monitored by the server loop and immediately terminates upon hitting either of these absolute state boundaries:

1. **Team Wipe:** All 3 characters assigned to a single player's active squad roster match-state simultaneously hold the KNOCKED_DOWN or DEAD flag.
2. **Player Surrender:** A client fires an explicit surrender action packet to the server interface.

5.2 Post-Game Character Recovery & Roster Cleaning

When a match ends, the server evaluates the state of all characters to process global database updates:

- **The Forfeit Safety Net:** If a player surrenders or disconnects, characters who are currently KNOCKED_DOWN or resting at low HP values are safe from permadeath. Their state flags reset to LIVING, and their health segments are healed to 100% capacity before they return to the lobby roster.
- **The Permanent Death Execution Routine:** If a character's state flag is marked as DEAD at the end of a match (meaning all 3 segments were disabled during play), run the structural cleanup routine:
 1. **Equipped Gear Deletion:** Query the character's unique slot pointers (main_hand_slot, off_hand_slot, armor_slot, and pocket items) and permanently delete those item entries from the database.

2. **Roster Erasure:** Disconnect the character's ID from the player's primary squad array, removing them entirely from their active roster list.
3. **The Hall of Fame Transition:** Extract the character's historic stats: Name, Class, achieved Level, and final base Stat values. Package this data into a minimized, read-only HallOfFameEntry object and append it to the player's persistent global profile record.
4. Free the active memory node assigned to that character entity.

5.3 Lobby Economy & Upgrades Pipeline

- **Match Rewards:** The server awards a fixed sum of Gold to a player's global profile account upon completing matches.
- **The Merchant Tab Interface:** Players can open the Merchant tab to spend Gold to buy new items or purchase fresh character templates. Character templates purchased from the merchant automatically populate with a basic default set of starting gear slots.
- **The Pending Level Up State:** When a character earns enough experience points to level up during a match, the server flags their status variable as PENDING_LEVEL_UP. The character is locked from entering new matches, and all stat choices or skill tree point allocations are held until the player opens their **Roster Tab** to manually assign them.

Phase 6: Comprehensive Verification & QA Test Scripts

To ensure your engine operates without bad interpretations, write automated unit tests using a framework like *GUT (Godot Unit Test)* to validate these specific edge cases:

- **Test Weight Bounds:** Verify that a character carrying items exactly 1kg over their limit loses 1 movement point, and carrying exactly 4kg over their limit loses 2 movement points.
- **Test Clamped Evasion Multipliers:** Force character dexterity modifiers to -5 and equipment armor buffs to -10. Verify that the evasion calculator clamps both individual components to 0 before summing them, preventing negative numbers from breaking accuracy logic.
- **Test Initiative Collisions:** Inject 3 characters with identical base initiative values. Run the initiative placement engine and verify that it resolves cleanly into three sequential, unique turn values without creating infinite loops.
- **Test Health Segment Spillover:** Create a character profile with three 50 HP health segments (150 Total HP). Apply a single damage hit of 75 HP. Verify that Segment 0 is completely destroyed, Segment 1 drops cleanly to 25 HP, and the character's state flag remains LIVING without triggering a knockdown state.